Xiangzhe Zeng

AI Agent Platform Engineer (Memory · RAG · Skill · Agent Runtime)

xz@xzos.net | +86 15530859511 | github.com/xz-dev | xzos.net

Summary

4 years of backend/full-stack experience. At Capital Dynamics I own the company's AI Agent infrastructure: I designed and shipped the Memory/RAG/Skill stack and a model gateway, and exposed those capabilities as services to the business. Previously I led the MCP tool platform at Klavis AI (the Strata engine: capability registration and orchestration). I use coding agents daily through Vibe Coding, maintain a Pi downstream distribution plus Memory/continuity/retry plugins, have source-level practice on Hermes Agent and Claude Code-class systems (agent loop, memory, and skills), and keep contributing upstream (see Open Source).
Engineering approach: baseline before migration (containerize the legacy system and add regression tests, then refactor); do not treat unknown outcomes as success (unreturned tool calls are marked "unknown effect" and are not replayed automatically); make boundaries and permissions explicit (filter models by user, use session IDs only for routing not authorization, allow only necessary network targets); fix the root cause and contribute upstream (reproduce in our system first, then send the fix upstream).

Open Source

- Agent Memory: pi-hermes-memory (Hermes-style memory plugin for Pi), 7 merged PRs, including atomic application of memory-merge plans; Honcho (7.9k stars, agent memory framework), merged a deriver startup health-check fix.
- Agent runtime / sub-agents: pi-subagents (4.1k stars), 7 merged PRs covering durable retrieval of async completion results, isolation of forked sub-sessions, and MCP request-header caching; Pi coding agent (119k stars), merged #6764 and #6697.
- MCP/Skill tool platform: Klavis-AI/klavis (5.8k stars), 34 merged PRs on the Strata engine and 20+ third-party MCP integrations (capability registration, retrieval, and orchestration).
- Model gateway: OmniRoute (84k stars), 57 PRs with 50 merged, covering overload control, model permissions, quota fallback, and inference streams; APISIX AISIX, 2 merged proxy-performance PRs; Sub2API (45k stars), 2 merged model-catalog PRs.
- Agent frontend: Hermes WebUI (19k stars), 2 merged PRs (session usage metering, reasoning-card state).
- Runtime fundamentals: Deno (108k stars) child-process signal fix and youki (Rust OCI runtime) both merged; virtio-win, 4 merged PRs (including RHEL-149886); ansible-runner #1306.
- Own projects: founder of UpgradeAll (1.3k+ stars); AI-gateway, 31 stars; Pi downstream, 16 stars.
- Totals: 368 PRs merged into other people's repositories since 2018 (GitHub search, 2026-11); 200+ public GitHub repositories.

Skills

Agents & LLM: agent loop/tool calling, context engineering (context compaction and continuity), agent memory (store/retrieve/manage/evolve: mem0, Honcho, Hindsight, Hermes-style memory), RAG retrieval and context construction (recall + semantic rerank), Skill/plugin registration and orchestration, sub-agents, MCP; daily use of Hermes Agent/Pi/Claude Code (Vibe Coding).
Gateways & protocols: APISIX (Lua), OpenAI/Anthropic-compatible APIs, SSE, WebSocket, gRPC, REST, OAuth 2.0, SSO/MFA.
Languages & data: Python (Django/DRF, FastAPI, asyncio), Go, TypeScript/Node.js/Bun, Rust, C#/.NET, Java/Spring Boot, C/C++; PostgreSQL, Redis/Celery, SQLite.
Deploy & operations: Docker/Podman, Kubernetes, Helm, KEDA, Ansible, GitHub Actions, Azure, GCP, Linux; OpenTelemetry/Jaeger/Loki/Grafana.

Experience

Capital Dynamics (Shanghai) | AI and Full-Stack Development Engineer 2026.05 – present

Own the company's AI Agent infrastructure and applications, and lead the Hong Kong investor-portal re-platforming; about 4 months: 1,388 commits and 122 MRs (105 merged).

- Agent Memory and RAG: built an LLM Wiki and a shared mem0-based memory store; designed a dual-memory architecture covering storage, retrieval, management, and evolution (Honcho for long-term profile and preferences, Hindsight for an entity/event graph that resolves omitted references); shipped a Thunderbird mail add-on to all staff with a hand-built RAG retrieval and context-construction pipeline (query planning → candidate-mail recall → LLM semantic rerank to answer), plus summaries, reply drafts, and deadline extraction; drafts only, no send permission requested.
- Agent Skills and research desk: built an investment research desk on the official Hermes Agent container; a Node WebUI proxies the OpenAI-compatible API server-side so the browser never sees keys; research state is a hierarchical knowledge graph that records the source of every change (user/AI); AI cannot overwrite user

content and may only add items or suggest changes; wrote a graph Skill contract (persistence only through the backend API) and combined Hermes built-in skills, memory, and session history for registration, retrieval, orchestration, and reuse; data persisted to disk; Given/When/Then behaviour tests.

- AI gateway, observability, and cost constraints: proposed a shared AI access layer and got an MVP boundary approved (gateway base + one local route + one approved cloud route), with data classification, audit, and GPU-capacity constraints; implemented an APISIX gateway with reversible privacy filtering (replace sensitive data before sending, restore it on the streamed response); deployed OpenTelemetry + Jaeger + Loki + Grafana, with the data plane on the internal network and the control plane isolated behind a WAF.
- Engineering governance review: as a reviewer, diagnosed a delayed recruiting platform as missing a governance loop rather than development capacity; recommended a black-box regression baseline before refactoring, and rolled out repo-level AGENTS.md plus shared AI-coding guidelines (to support team Vibe Coding).
- Portal re-platforming: containerized the legacy PHP + MySQL system and added behavioural regression tests as a baseline, then migrated to React + Django/DRF (PostgreSQL, Redis/Celery, SSO, API gateway); shipped login, holdings, trading, and the document center in phases and owned ongoing maintenance of the production release; wrote data-validation, cut-over, and rollback plans.
- Data and authorization: holdings-import approval uses transactions, row locks, and content fingerprints for idempotent imports; approval is bound to the data version the reviewer saw so stale data cannot be approved; end-to-end reconciliation tests from source files to reports; split SSO token issuance, MFA, and service authorization, and fixed account-ownership checks to prevent object-ID privilege escalation; dual-review publishing in the document center, with authenticated downloads from private storage.

Klavis AI | Full-Stack Engineer / MCP Platform Architect 2025.07 – 2026.01

- Led the Strata engine, turning OpenAPI v2/v3 specs into agent-callable MCP tools/Skills and exposing them as platform services to the business; a prebuilt object cache plus reference-resolution optimizations cut complex-spec load time from 15 minutes $\rightarrow$ 3 seconds.
- Built and maintained 20+ third-party integrations (Google Workspace, PayPal, Sentry, and others) with OAuth authorization and token refresh; moved Playwright sessions into separate processes with timeout cleanup; fixed concurrent HTTP-client reuse in Google Calendar.
- Kubernetes/Helm private deployment; KEDA starts services on request and scales idle ones to zero, running hundreds of tool services on a $\sim$4GB-RAM VPS (cost and elasticity); GitHub Actions builds, scans, and ships multi-arch images.

Microsoft Office 365 Interoperability and Compliance Project | Full-Stack Engineer 2023.10 – 2025.07

- Python Word-compatibility test platform (Windows APIs, OCR, VM scheduling) running 800+ cases weekly, cutting manual testing time by about 50%; tested 400 Microsoft Graph API endpoints in 20 days with Copilot and MCP tools.
- A GB-standard compliance document checker that edits Word Open XML directly, cutting per-person review-and-fix time from 2.5 hours $\rightarrow$ 5 minutes.
- Refactored the document-publishing tool onto Azure Pipelines, processing 30GB/400k files weekly and saving about 3 person-days per release; switched to Azure Managed Identity.
- Migrated scheduled jobs from a .NET Framework monolith to containerized .NET services, cutting Azure cost by 85% and scheduling time by 50%; added CodeQL to the pipeline.

Talkweb Information | Java Backend Developer 2022.06 – 2023.09

- Backend for China Unicom's A-share portal and ticketing system; refactored Spring Boot microservices, Kubernetes auto-deploy and scaling, and IPFS-based file storage.

Personal Open Source

Pi downstream and plugins | coding agent, downstream maintainer 2026.06 – present

- Context engineering: moved context compaction to before the request is sent so user input and compaction records keep a correct save order; task continuity and structured compaction plugin; after a failed request, continue from the last protocol-complete point on a new branch, keep the failure record, and mark unreturned tool calls as "unknown effect".
- Agent loop / runtime: async tools support background execution, cancellation, and result retrieval; fixed sessions hanging after Esc; added PTY and tool-batch regression tests; model-catalog refresh has a timeout and falls back to a local snapshot.
- Skill / plugin ecosystem: cross-process collaboration library (HMAC handshake, message acknowledgement), retry (do not retry quota/billing errors), notify; Bun-packed cross-platform single binary, self-update with verification and atomic replace after backup; patches on independent branches, with CI tracking upstream.

AI-gateway | multi-model gateway, independently designed and built 2026.08 – present

- Ingress, auth/billing, and model invocation use existing components (APISIX and others); I wrote the Go/Rust/Lua services in between: a Go model catalog and a Rust sync tool that collect upstream models and capabilities; APISIX Lua filters the model list by user permission (multi-tenant isolation) while keeping full metadata.

- Go streaming proxy: keep-alive during long reasoning in each of OpenAI/Anthropic's SSE formats, proxy WebSocket, and report handshake failures and mid-stream errors back to the client; session IDs are passed to the routing layer for sticky routing and are used only for routing, not authorization.
- Inter-container and egress networks allow only necessary targets; Ansible plus pinned image digests for deploys; tests for permission filtering, streaming errors, session routing, and network boundaries.

UpgradeAll | Android app-update manager, project founder 2019.04 – present

- Led a six-person team aggregating update sources such as GitHub and F-Droid (1.3k+ stars); owned the Kotlin client, Rust update core, and the early Python server (gRPC/REST, Redis, Docker); in 2026 completed the AGP 9/Gradle 9 migration, unified Kotlin–Rust calls over WebSocket JSON-RPC, and landed upstream fix PR#14.

Education & Certifications

North China University of Science and Technology | B.Eng., Computer Science and Technology 2023.06
Red Hat Certified Engineer RHCE (285/300); RHCSA (full marks); RISC-V Foundational Associate (RVFA). ASC18 Student Supercomputer Challenge, second prize; selected for the Linux Foundation talent incentive program.